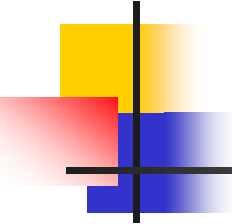


算法设计与分析

Analysis and Design of Algorithm

Lesson 10



要点回顾

- 贪心算法的基本思想
 - 求解最优化问题的算法包含一系列步骤
 - 每一步都有一组选择
 - 作出在当前看来最好的选择
 - 希望通过作出局部最优选择达到全局最优选择
 - 贪心算法不一定总产生最优解
- 贪心算法产生最优解的条件
 - 最优子结构性质
 - 贪心选择性质
- 几个例子
 - 找零钱、一般背包问题、活动安排/选择问题

活动选择问题

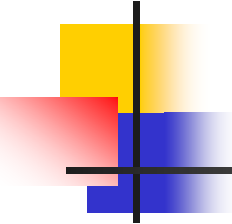
- **输入：** $S=\{1,2,\dots,n\}$ 为 n 项活动的集合， s_i, f_i 分别为活动 i 的开始和结束时间。

活动 i 与 j 相容 $\leftrightarrow s_i \geq f_j$ 或 $s_j \geq f_i$

- **输出：** 最大的两两相容的活动集 A
- **实例：**

i	1	2	3	4	5	6	7	8	9	10
s_i	1	3	2	5	4	5	6	8	8	2
f_i	4	5	6	7	9	9	10	11	12	13

解： $A=\{1,4,8\}$



贪心算法

挑选过程是多步判断过程，每步依据某种“短视”的策略进行活动的选择，选择时候需要满足相容性条件。

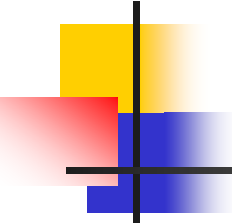
- 策略1：开始时间早的优先
 - 排序使 $s_1 \leq s_2 \leq \dots \leq s_n$ ，从前往后挑选
- 策略2：占用时间少的优先
 - 排序使 $f_1 - s_1 \leq f_2 - s_2 \leq \dots \leq f_n - s_n$ ，从前往后挑选
- 策略3：结束时间早的优先
 - 排序使 $f_1 \leq f_2 \leq \dots \leq f_n$ ，从前往后挑选

活动选择贪心策略3伪码

- 算法 GreedySelect
- 输入：活动集 S , $s_i, f_i, i=1,2,\dots,n, f_1 \leq f_2 \leq \dots \leq f_n$
- 输出： $A \subseteq S$, 选中的活动子集
 1. $n \leftarrow \text{length}[S]$
 2. $A \leftarrow \{1\}$
 3. $j \leftarrow 1$
 4. for $i \leftarrow 2$ to n do
 5. if $s_i \geq f_j$
 6. then $A \leftarrow A \cup \{i\}$
 7. $j \leftarrow i$
 8. return A

如何证明策略3的正确性？

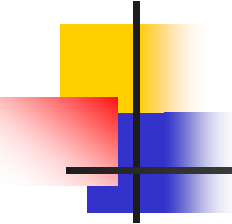
完成时间 $t = \max \{f_k : k \in A\}$



贪心算法正确性归纳证明

■ 证明步骤

1. 叙述一个有关自然数 n 的**命题**，该命题断定该贪心策略的执行最终将导致最优解。其中自然数 n 可以代表步数或问题规模
2. 证明命题对所有的自然数为真
 - **2.1 归纳基础**（从最小实例规模开始）
 - **2.2 归纳步骤**（第一或第二数学归纳法）



活动选择算法的命题

1. 命题：

算法GreedySelect执行到第 k 步，选择 k 项活动

$i_1, i_2, \dots, i_k$

则存在最优解 A 包含活动 $i_1=1, i_2, \dots, i_k$ 。

根据上述命题：对于任何 k ，算法前 k 步的选择都将导致最优解，至多到第 n 步将得到问题实例的最优解。

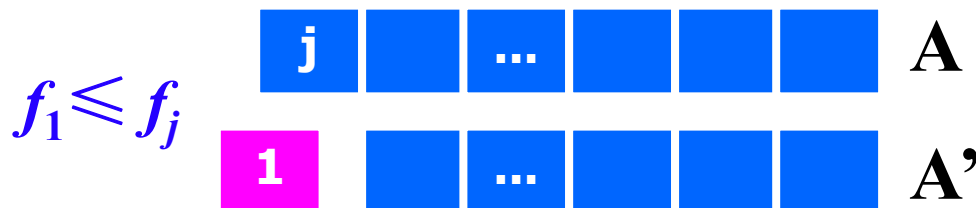
归纳法证明

令 $S=\{1,2,\dots,n\}$ 是活动集，且 $f_1 \leq f_2 \leq \dots \leq f_n$

■ **2.1 归纳基础：** $k=1$, 证明存在最优解包含 **活动1**

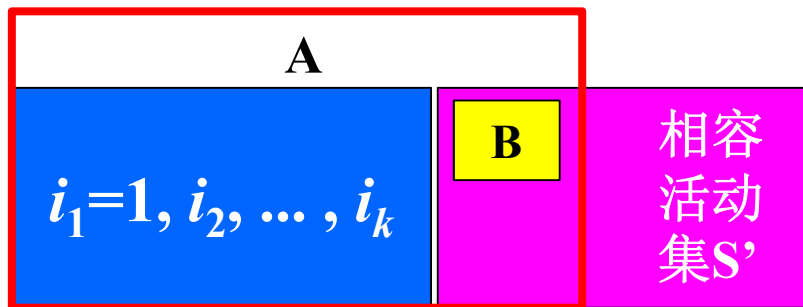
■ **证明：**

- 任取最优解 A ， A 中活动按结束时间递增排序。如果 A 的第一个活动为 j ，且 $j \neq 1$ ，用 1 替换 A 的活动 j 得到解 A' ，即 $A'=(A-\{j\}) \cup \{1\}$
- 由于 $f_1 \leq f_j$ ， A' 也是最优解，且含有 1。

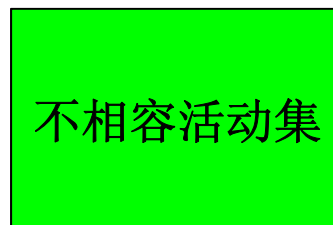


归纳法证明

- **2.2 归纳步骤：** 假设命题对 k 为真，证明对 $k+1$ 也为真
- **证明：**
 - 算法执行到第 k 步，选择了活动 $i_1, i_2, \dots, i_k$ ，根据归纳假设存在最优解 A 包含 $i_1=1, i_2, \dots, i_k$ ，且 A 中剩下活动 B 选自集合 S'
 - $S' = \{ i \mid i \in S, s_i \geq f_k \}$ ， $A = \{i_1=1, i_2, \dots, i_k\} \cup B$



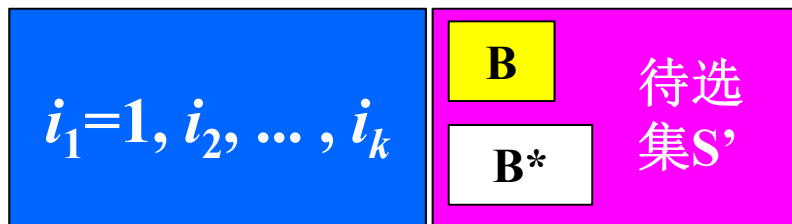
B和 S' 存在什么关系？



归纳步骤(cont.)

- **B是S'的最优解。**
 - 若不然，S'的最优解为B*，B*的活动比B多，那么

$B^* \cup \{i_1=1, i_2, \dots, i_k\}$
是S的最优解，且比A的活动多，与A的最优性矛盾！

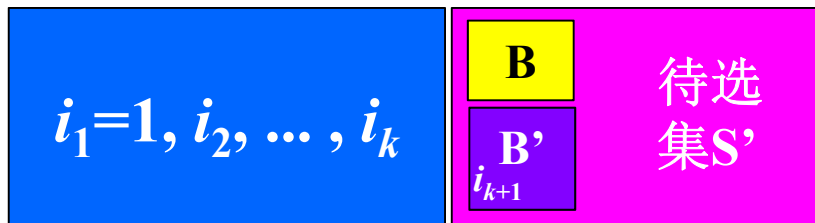


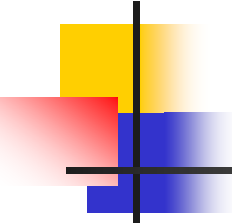
归纳步骤(cont.)

- 将 S' 看成子问题，根据归纳基础，存在 S' 的最优解 B' 有 S' 中的第一个活动 i_{k+1} ，且 $|B'|=|B|$ ，于是：

$$\begin{aligned} & \{i_1=1, i_2, \dots, i_k\} \cup B' \\ &= \{i_1=1, i_2, \dots, i_k, i_{k+1}\} \cup (B' - i_{k+1}) \end{aligned}$$

也是原问题的最优解。





证明方法小结

- 贪心法正确性证明方法：数学归纳法
 - 第一数学归纳法
 - 第二数学归纳法
- 活动选择问题的贪心法证明：
 - 叙述一个涉及步数的算法正确性命题
 - 证明归纳基础
 - 证明归纳步骤

最优装载问题

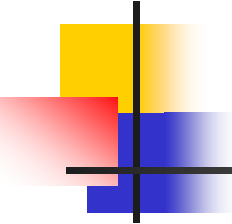
- **问题：** 有 n 个集装箱 $1, 2, \dots, n$ 装上轮船，集装箱 i 的重量 w_i ，轮船装载重量限制为 C ，无体积限制。问如何装，使得船上的集装箱的数目最多？不妨设每个箱子的重量 $w_i \leq C$



最优装载问题

- 该问题是**0-1背包问题的子问题**。集装箱相当于物品，物品的重量是 w_i ，价值 v_i 都等于1，轮船载重限制C相当于背包的重量限制b。





问题建模

- 设 $\langle x_1, x_2, \dots, x_n \rangle$ 是解向量，其中 $x_i \in \{0, 1\}$ ， $x_i = 1$ 当且仅当第 i 个集装箱上船

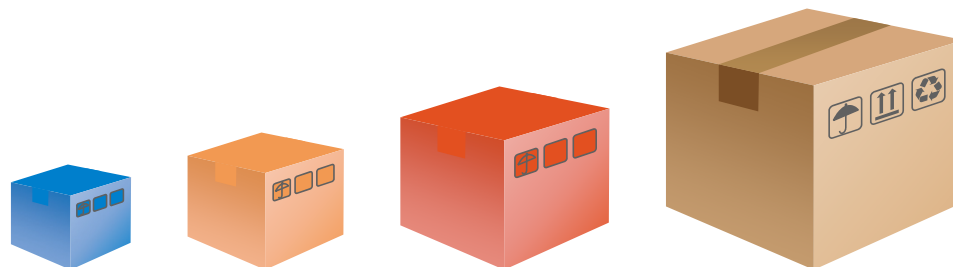
目标函数 $\max \sum_{i=1}^n x_i$

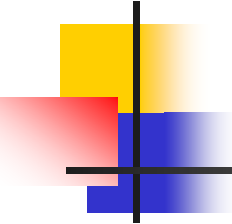
约束条件 $\sum_{i=1}^n w_i x_i \leq C$

$$x_i \in \{0, 1\} \quad i=1, 2, \dots, n$$

算法设计

- 贪心策略G: 轻者优先
- 算法设计:
 - 将集装箱排序, 使得 $w_1 \leq w_2 \leq \dots \leq w_n$
 - 按照标号从小到大装箱, 直到装入下一个箱子将使得集装箱总重超过轮船的装载重量限制, 则停止。





正确性证明思路

- **命题：**对于装载问题任何规模为 n 的输入实例，贪心算法G得到最优解。
- 设集装箱从轻到重记为 $1, 2, \dots, n$
- **归纳基础** 证明对任何只含1个箱子的输入实例，贪心法得到最优解。显然正确！
- **归纳步骤** 证明：假设对于任何 n 个箱子的输入实例贪心法都能得到最优解，那么对于任何 $n+1$ 个箱子的输入实例贪心法也得到最优解。

归纳步骤证明思路

01

$N = \{ 1, 2, \dots, n, n+1 \},$
 $w_1 \leq w_2 \leq \dots \leq w_{n+1}$

02

去掉箱子1, 令 $C' = C - w_1$
得到规模为 n 的输入 $N' = \{ 2, 3, \dots, n, n+1 \}$

03

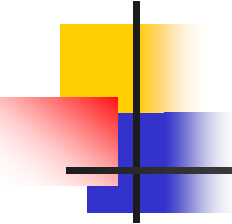
关于输入 N' 和 C' 的最优解 I'

04

在 I' 加入箱子1, 得到 I

05

证明 I 是关于输入 N 的最优解



正确性证明

- 假设对于 n 个集装箱的输入，贪心法都可以得到最优解，考虑输入

$$N = \{ 1, 2, \dots, n, n+1 \}$$

其中 $w_1 \leq w_2 \leq \dots \leq w_{n+1}$

- 由归纳假设，对于

$$N' = \{ 2, 3, \dots, n, n+1 \}, \quad C' = C - w_1$$

- 贪心法得到最优解 I' 。令

$$I = I' \cup \{ 1 \}$$

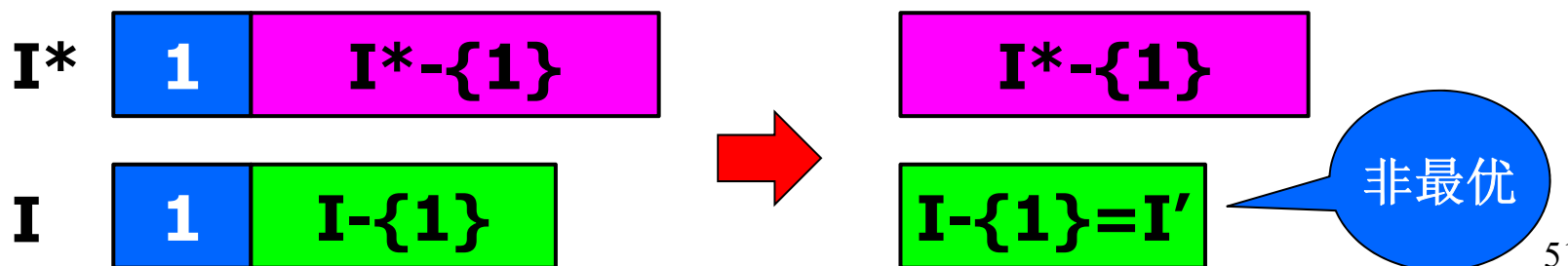
正确性证明

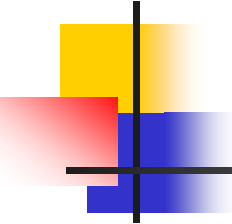
- **I(算法解)是关于N的最优解**

- 若不然，存在包含1的关于N的最优解 I^* (如果 I^* 中没有1，用1替换 I^* 中的第一个元素得到的解也是最优解)，且 $|I^*| > |I|$ ；那么 $I^* - \{1\}$ 是 N' 和 C' 的解且

$$|I^* - \{1\}| > |I - \{1\}| = |I'|$$

与 I' 是关于 N' 和 C' 的最优解矛盾。



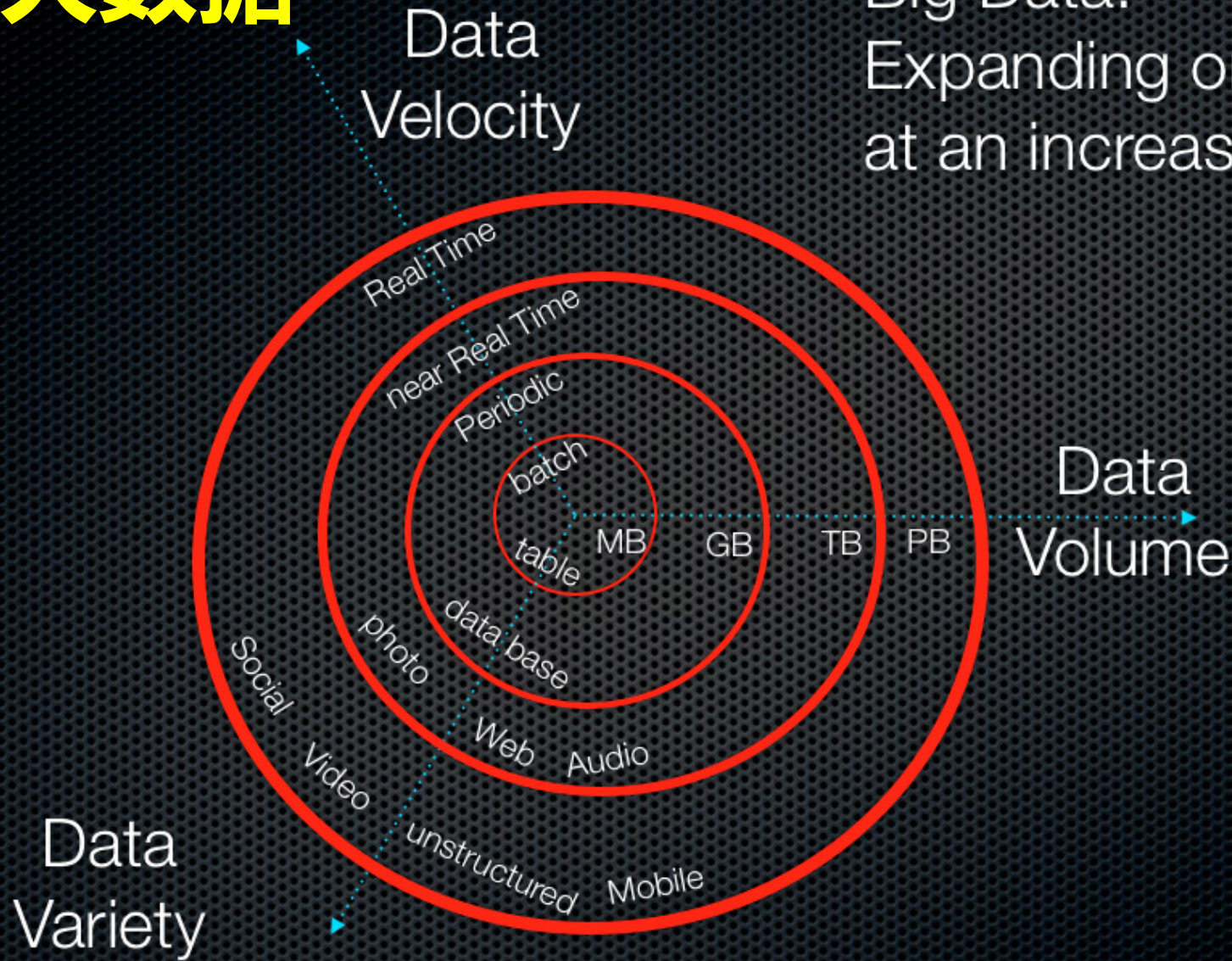


小结

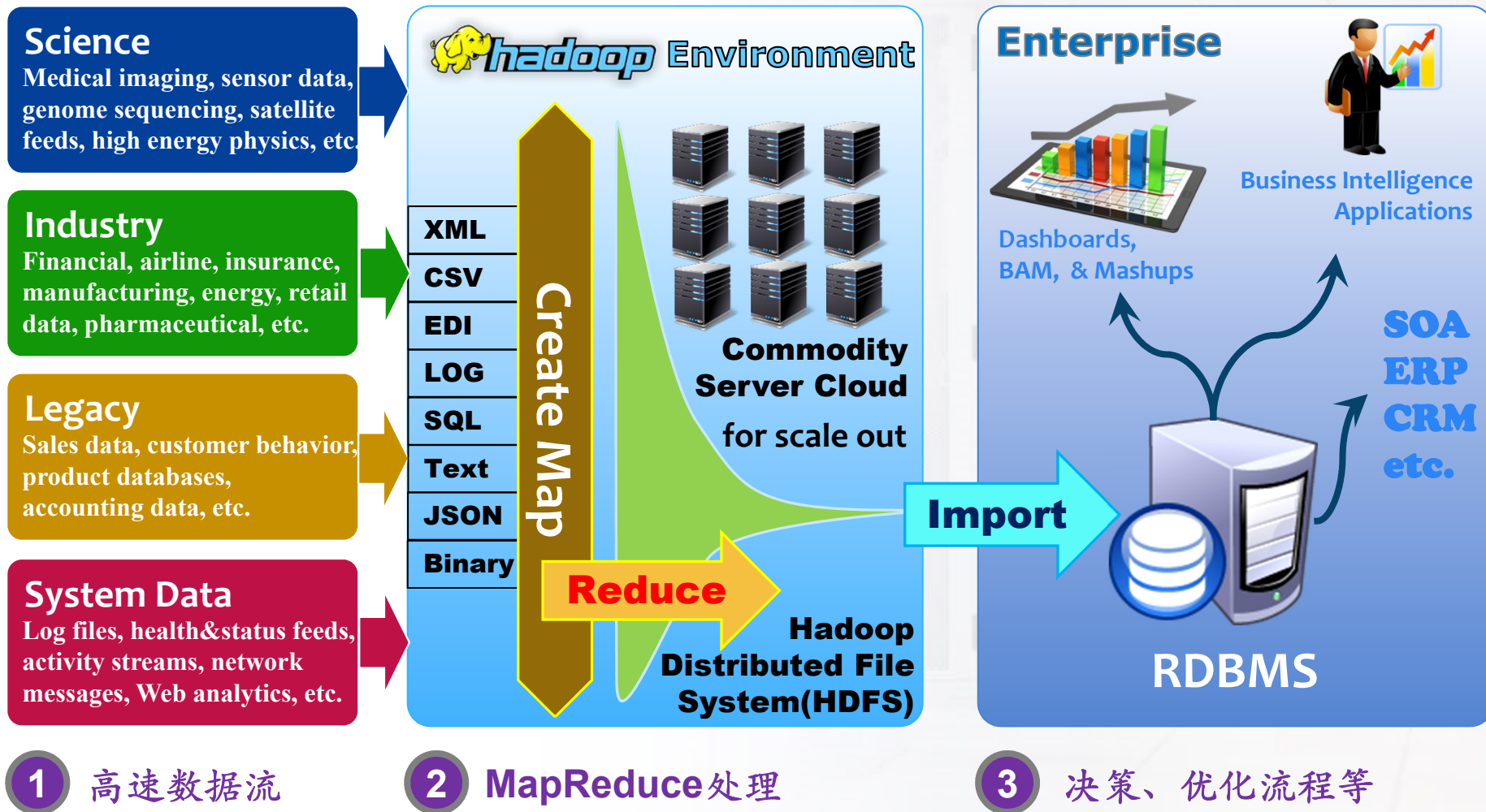
- 装载问题是**0-1背包问题的子问题**（每件物品价值为1），NP难的问题存在多项式时间可解的子问题。
- 贪心法证明：对**规模**进行归纳

大数据

Big Data:
Expanding on 3 fronts
at an increasing rate.

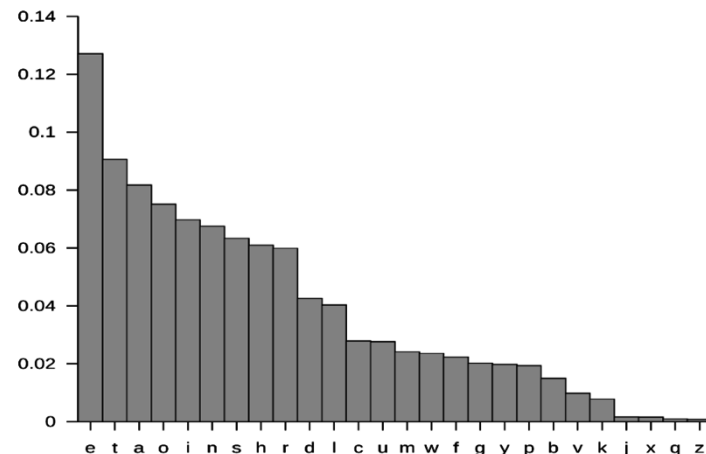
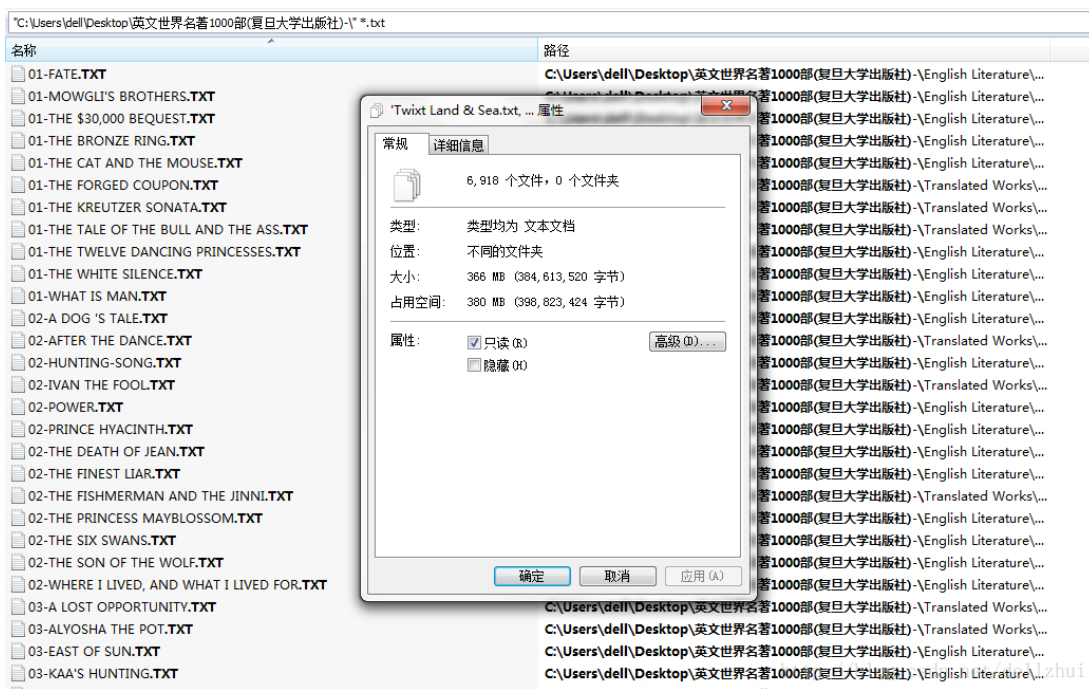


大数据处理代表框架Hadoop



一个有趣的大数据统计

- 26英文字母使用频率的大样本分析
 - 样本：英文世界名著（1000部）
 - 样本容量：1.4亿（包含6918个文件，366M）



二元前缀码及其应用

- 二元前缀码是广泛用于数据**文件压缩**的编码方法，其使用字符在文件中出现的频率表来建立一个用0,1串表示各个字符的最优表示方式。



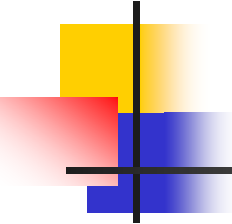
**Draco 3D
compressor**

facebook

Zstandard



pied piper



二元前缀码

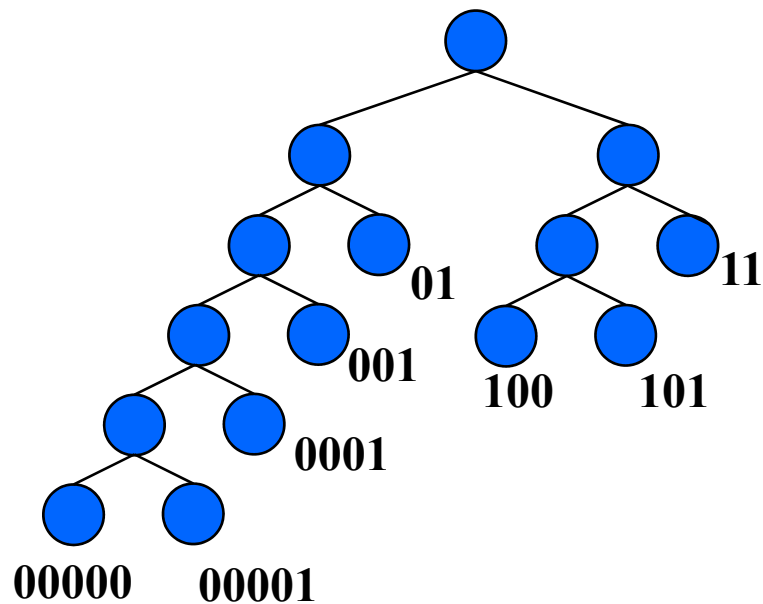
- 用0-1字符串作为代码表示字符，要求任何字符的代码都不能作为其他字符代码的前缀
- 非前缀码的例子

a: **001**, b: **00**, c: **010**, d: **01**

- 解码的歧义，例如字符串**0100001**
 - 解码1: 01, 00, 001 d, b, a
 - 解码2: 010, 00, 01 c, b, d

$$\{00000, 00001, 0001, 001, 100, 101, 11, 01\}$$

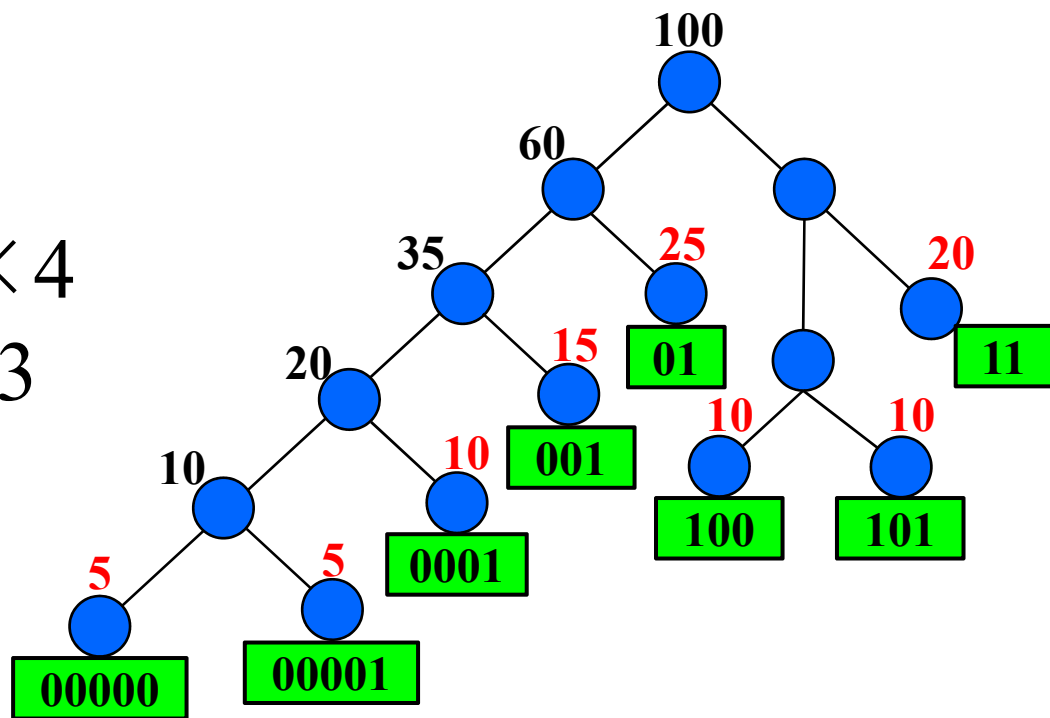
- 0-左子树
- 1-右子树
- 码对应一片树叶
- 最大位数为树的层数



平均传输位数

$$B = \sum_{i=1}^n f(x_i) d(x_i)$$

$$\begin{aligned} B &= [(5+5) \times 5 + 10 \times 4 \\ &\quad + (15+10+10) \times 3 \\ &\quad + (25+20) \times 2] \\ &\quad \div 100 \\ &= 2.85 \end{aligned}$$



问题： 给定字符集 $C=\{x_1, x_2, \dots, x_n\}$ 和每个字符的频率 $f(x_i)$, $i=1, 2, \dots, n$ 。求关于 C 的一个**最优前缀码**（平均传输位数最小）。

哈夫曼树算法伪码

- 算法 Huffman(C)
- 输入: $C = \{x_1, x_2, \dots, x_n\}, f(x_i), i=1, 2, \dots, n$
- 输出: Q //队列
 1. $n \leftarrow |C|$
 2. $Q \leftarrow C$ //频率递增队列Q
 3. for $i \leftarrow 1$ to $n-1$ do
 4. $z \leftarrow \text{Allocate-Node}()$ //生成结点z
 5. $z.\text{left} \leftarrow Q$ 中最小元 //最小作z左儿子
 6. $z.\text{right} \leftarrow Q$ 中最小元 //最小作z右儿子
 7. $f(z) \leftarrow f(x) + f(y)$
 8. Insert(Q, z) //将z插入Q
 9. return Q

实例

Q: $f:5, e:9, c:12, b:13, d:16, a:45$

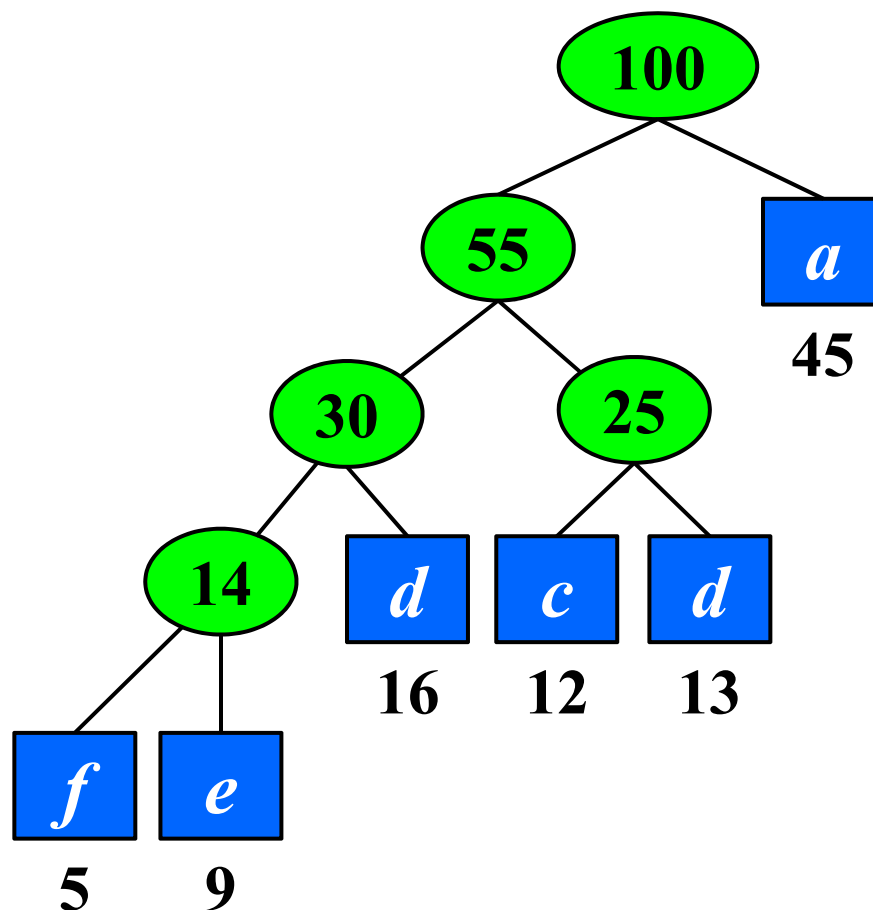
输入: $a:45, b:13, c:12, d:16, e:9, f:5$

编码:

- $f \rightarrow 0000$
- $e \rightarrow 0001$
- $d \rightarrow 001$
- $c \rightarrow 010$
- $b \rightarrow 011$
- $a \rightarrow 1$

平均位数?

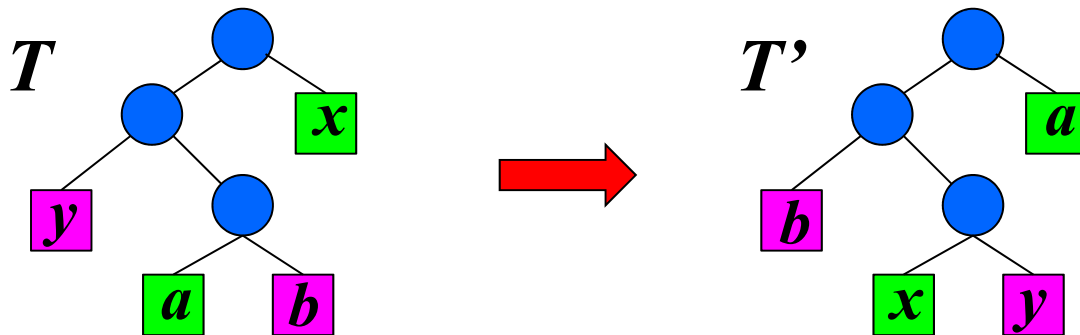
如何证明正确性?



最优前缀码性质

引理1: C 是字符集, $\forall c \in C, f(c)$ 为频率, $x, y \in C, f(x), f(y)$ 频率最小, 那么存在最优二元前缀码, 使得 x, y 码字等长且仅在最后一位不同。

$$\begin{aligned} f(x) &\leq f(a) \\ f(y) &\leq f(b) \end{aligned}$$



$$B(T) - B(T') = \sum_{i \in C} f(i) d_T(i) - \sum_{i \in C} f(i) d_{T'}(i) \geq 0$$

其中 $d_T(i)$ 为 i 在 T 中的层数(i 到根的距离)

最优前缀码性质

引理2: 设 T 是二元前缀码的二叉树, $\forall x, y \in T$, x, y 是树叶兄弟, z 是 x, y 的父亲, 令

$$T' = T - \{x, y\}$$

且令 z 的频率

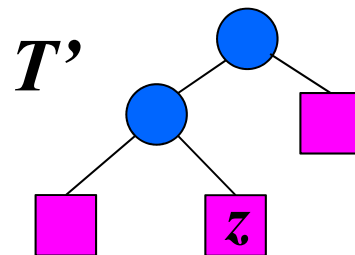
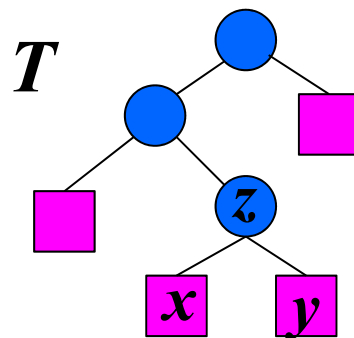
$$f(z) = f(x) + f(y)$$

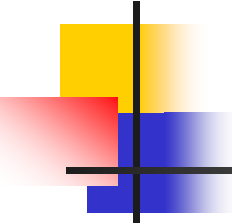
T' 是对应二元前缀码

$$C' = (C - \{x, y\}) \cup \{z\}$$

的二叉树, 那么

$$B(T) = B(T') + f(x) + f(y)$$





引理2证明

证明: $\forall c \in C - \{x, y\}$, 有

$$d_T(c) = d_{T'}(c) \Rightarrow f(c)d_T(c) = f(c)d_{T'}(c)$$

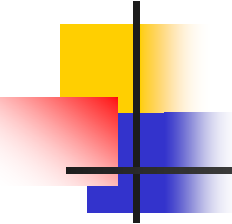
$$d_T(x) = d_T(y) = d_{T'}(z) + 1$$

$$B(T) = \sum_{i \in T} f(i)d_T(i)$$

$$= \sum_{i \in T, i \neq x, y} f(i)d_T(i) + f(x)d_T(x) + f(y)d_T(y)$$

$$= \sum_{i \in T', i \neq z} f(i)d_{T'}(i) + f(z)d_{T'}(z) + (f(x) + f(y))$$

$$= B(T') + f(x) + f(y)$$



算法正确性证明思路

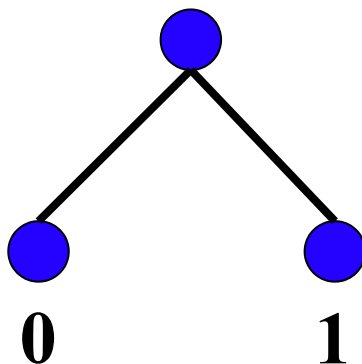
定理 Huffman算法对任意规模为 $n(n \geq 2)$ 的字符集 C 都得到关于 C 的最优前缀码的二叉树。

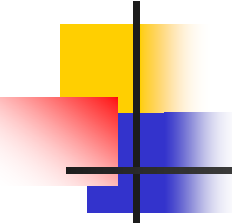
归纳基础 证明：对于 $n=2$ 的字符集，Huffman算法得到最优前缀码。

归纳步骤 证明：假设Huffman算法对于规模为 k 的字符集都得到最优前缀码，那么对于规模为 $k+1$ 的字符集也得到最优前缀码。

归纳基础

- $n=2$, 字符集 $C=\{x_1, x_2\}$
- 对任何代码的字符至少都需要1位二进制数字。Huffman算法得到的代码是0和1, 是最优前缀码。





归纳步骤

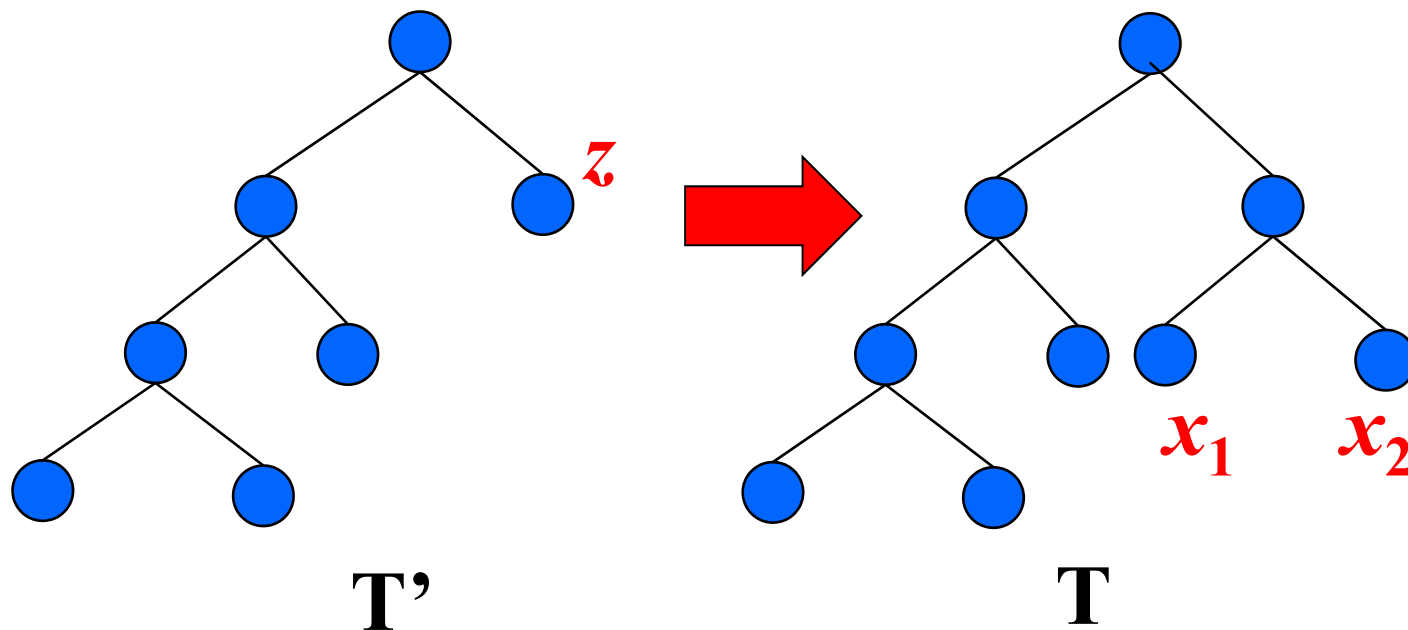
- 假设Huffman算法对于规模为 k 的字符集都得到最优前缀码。考虑规模为 $k+1$ 的字符集

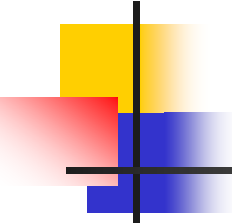
$$C = \{x_1, x_2, \dots, x_{k+1}\}$$

- 其中 x_1, x_2 是 C 中频率最小的两个字符。
- 令， $C' = (C - \{x_1, x_2\}) \cup \{z\}$, $f(z) = f(x_1) + f(x_2)$
- 根据归纳假设，算法可以得到一棵关于字符集 C' ，频率 $f(z)$ 和 $f(x_i)$ ($i=3, 4, \dots, k+1$)的最优前缀码的二叉树 T'

归纳步骤(cont.)

- 把 x_1, x_2 作为 z 的儿子附到 T' 上, 得到树 T , 那么 T 是关于 $C=(C'-\{z\}) \cup \{x_1, x_2\}$ 的最优前缀码的二叉树。





归纳步骤(cont.)

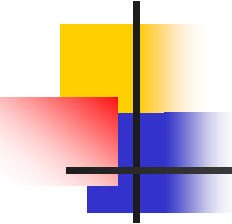
- 若不然,存在更优树 T^* , $B(T^*) < B(T)$, 且由引理1, 其树叶兄弟是 x_1 和 x_2 。

- 去掉 T^* 中的 x_1 和 x_2 , 得到 $T^{*'}$ 。根据引理2

$$\begin{aligned} B(T^{*'}) &= B(T^*) - (f(x_1) + f(x_2)) \\ &< B(T) - (f(x_1) + f(x_2)) \\ &= B(T') \end{aligned}$$

- 与 T' 是一棵关于 C' 的最优前缀码的二叉树矛盾。

图问题中的贪心算法



最小生成树(Minimum Spanning Tree)

无向连通带权图

$$G=(V,E,W)$$

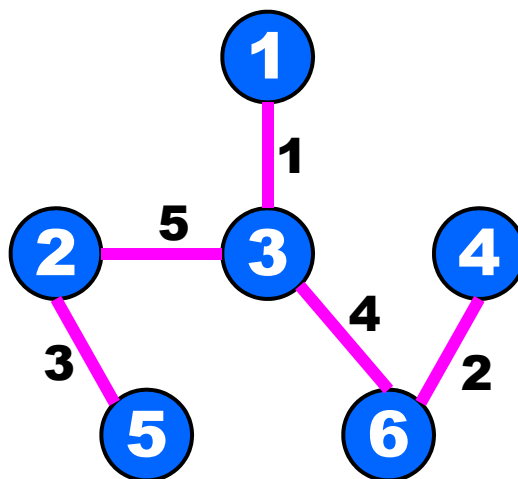
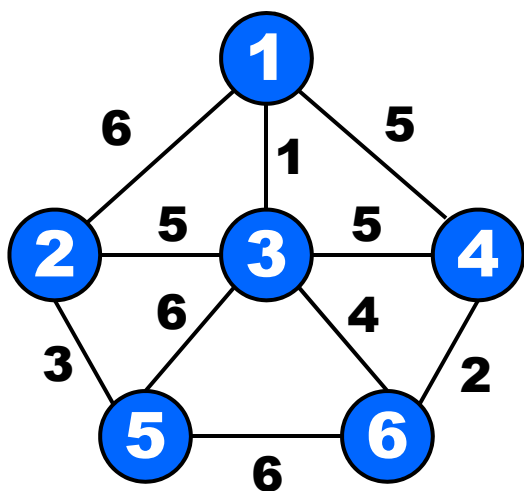
其中 $w(e) \in W$ 是边 e 的权值。

G 的一棵生成树 T 是包含了 G 所有顶点的树，树中各边的权值之和 $W(T)$ 称为树的权，具有最小权的生成树称为 G 的最小生成树。

最小生成树的实例

$G=(V, E, W)$, $V=\{1,2,3,4,5,6\}$, W 如图所示。

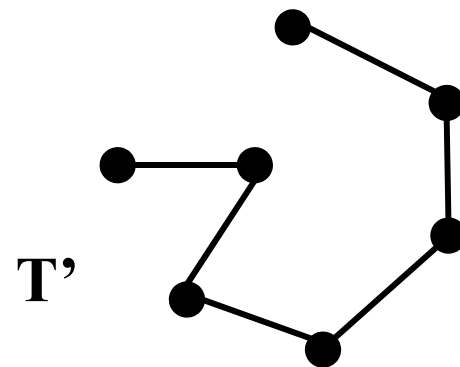
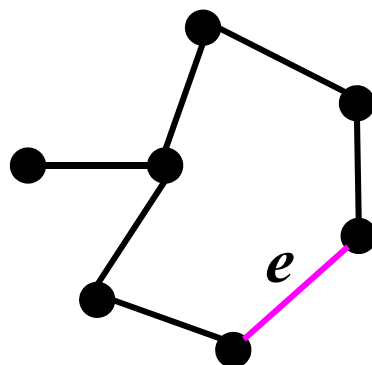
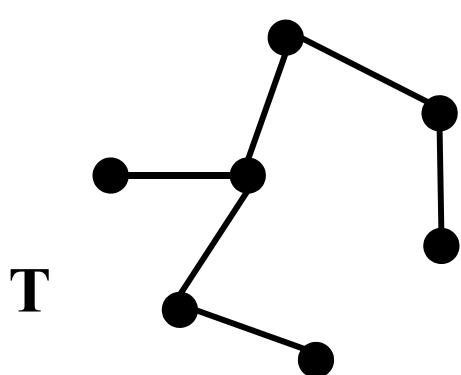
$E=\{\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,5\},$
 $\{3,4\},\{3,5\},\{3,6\},\{4,6\},\{5,6\}\}$

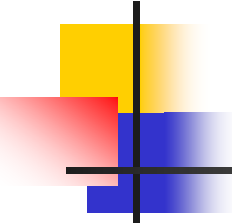


生成树的性质

命题1: 设 G 是 n 阶连通图, 那么

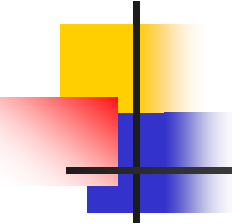
1. T 是 G 的生成树当且仅当 T 无圈且有 $n-1$ 条边;
2. 如果 T 是 G 的生成树, $e \notin T$, 那么 $T \cup \{e\}$ 含有一个圈 C (回路)
3. 去掉圈 C 的任意一条边, 就得到 G 的另外一棵生成树 T'





生成树性质的应用

- 算法步骤：**选择边**。
约束条件：不形成回路
截止条件：边数达到 $n-1$
- **改进生成树T的方法**
在T中加一条非树边 e ，形成回路C，在C中去掉一条树边 e' ，形成一棵新的生成树T'
$$W(T') - W(T) = W(e) - W(e')$$
若 $W(e) \leq W(e')$ ，则 $W(T') \leq W(T)$



求最小生成树

- **问题：**

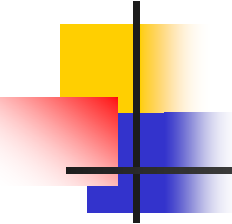
给定连通带权图 $G=(V,E,W)$, $W(e) \in W$ 是边 e 的权。求 G 的一棵最小生成树。

- **贪心法：**

- Prim算法

- Kruskal算法

最小生成树有着重要应用！



Prim算法

■ 设计思想

- 输入：图 $G=(V,E,W)$, $V=\{1,2,\dots,n\}$
- 输出：最小生成树 T
- 步骤：
 - 初始 $S=\{1\}$;
 - 选择连接 S 与 $V-S$ 集合的最短边 $e=\{i, j\}$, 其中 $i \in S$, $j \in V-S$ 。将 e 加入树 T , j 加入 S ;
 - 继续执行上述过程, 直到 $S=V$ 为止。

Prim算法伪码

■ 算法Prim(G,E,W)

1. $S \leftarrow \{1\}$
2. **while** $V-S \neq \emptyset$ **do**
3. 从 $V-S$ 中选择 j 使得 j 到 S 中顶点的边权最小
4. $S \leftarrow S \cup \{j\}$

■ 实例:

